

Opakování jazyka C

Co z druhého ročníku využijeme na mikrokontroleru
Programování – 3. ročník (Mechatronika)

Jaroslav Bušek

Střední průmyslová škola na Proseku

Co dnes zvládneme

- Zopakují si jazyk C **z pohledu mikrokontroleru** – ne všechno, jen to, co bude celý rok potřeba.
- Projdu datové typy, větvení, cykly, funkce a práci s pamětí a u každého si řeknu, **co platí na Arduinu jinak než na PC**.
- Poznám pět míst, kde se dá na mikrokontroleru snadno „spadnout“.

Jak to bude: u každé otázky si nejdřív **tipnete** (A–D, nahlas nebo rukou), pak se odpověď odkryje. Za špatný tip se nic nestane – právě ty jsou nejužitečnější.

Druhý ročník vs. třetí ročník

Jazyk zůstává stejný. Proměnné, if, cykly, pole, funkce — nic z toho se nebude učit znovu.

Mění se stroj, na kterém to běží:

- žádný operační systém
- žádná klávesnice ani obrazovka
- paměť v **kilobajtech**, ne v gigabajtech
- program **nikdy neskončí**
- chyba se neohlásí – deska se „zasekne“ nebo restartuje

obrázek k dodání

Srovnání vedle sebe: notebook (OS, klávesnice, obrazovka, GB RAM) a Arduino UNO (piny, žádný OS, 2 kB SRAM)

`../obrazky/01_pc_vs_mcu`

Datové typy a rozsahy

Kvíz 1 – kolik bajtů má `int`?

Stejný řádek přeložíme jednou pro PC a jednou pro Arduino UNO:

```
printf("%d", sizeof(int)); // na PC  
Serial.println(sizeof(int)); // na Arduino UNO
```

- A) Obojí vypíše 4 – `int` je vždycky 4 bajty.
- B) PC vypíše 4, Arduino 2.
- C) PC vypíše 2, Arduino 4.
- D) Nelze říct, `sizeof` vrací počet bitů.

Kvíz 1 – kolik bajtů má `int`?

Stejný řádek přeložíme jednou pro PC a jednou pro Arduino UNO:

```
printf("%d", sizeof(int)); // na PC  
Serial.println(sizeof(int)); // na Arduino UNO
```

- A) Obojí vypíše 4 – `int` je vždycky 4 bajty.
- B) PC vypíše 4, Arduino **2**.
- C) PC vypíše 2, Arduino 4.
- D) Nelze říct, `sizeof` vrací počet bitů.

B)

Velikost `int` určuje **překladač pro danou platformu**. Na AVR jsou to 2 bajty, takže `int` umí jen **-32 768 až 32 767**.

Proto se píší typy s číslem

Typ	B	Rozsah
uint8_t	1	0 ...255
int8_t	1	-128 ...127
uint16_t	2	0 ...65 535
int16_t	2	-32 768 ...32 767
uint32_t	4	0 ...4 294 967 295
float	4	~7 číslic

byte = uint8_t, word = uint16_t.
Arduino názvy, ale radši ty s číslem.

```
// nejasne: kolik to ma bitu?  
int pin = 13;  
int cas = 100000;    // preteklo!  
  
// jasne na prvnim pohled:  
uint8_t pin = 13;  
uint32_t cas = 100000;
```

Pravidlo pro celý rok: číslo pinu a malé počty do uint8_t, čas v milisekundách do uint32_t.

Kvíz 2 – co se vypíše?

```
uint8_t pocet = 250;

void setup() {
  Serial.begin(9600);
  for (uint8_t i = 0; i < 10; i++) {
    pocet = pocet + 1;    // desetkrát
  }
  Serial.println(pocet);
}
```

- A) 260 B) 255 C) 4 D) program zahlásí chybu

Kvíz 2 – co se vypíše?

```
uint8_t pocet = 250;

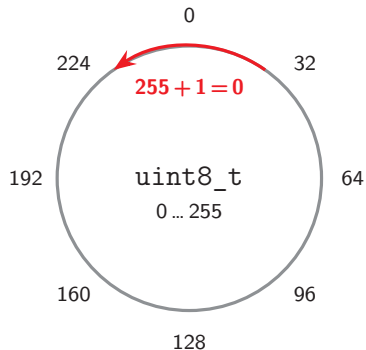
void setup() {
  Serial.begin(9600);
  for (uint8_t i = 0; i < 10; i++) {
    pocet = pocet + 1;    // desetkrát
  }
  Serial.println(pocet);
}
```

- A) 260 B) 255 **C) 4** D) program zahlásí chybu

C) – uint8_t po 255 přeteče na nulu

250 → 255 → **0** → 1 → ... → 4. Žádné varování – číslo se „přetočí“ jako počítadlo kilometrů.

Přetečení si představujte jako ciferník



Kde to letos potkáme

- počítadlo stisků tlačítka v `uint8_t`
- součet hodnot z ADC (0–1023) do `int` – přeteče po 32 vzorcích
- `millis()` po 49 dnech běhu

Dobrá zpráva: u typů unsigned je přetečení **definované chování**, ne chyba. V bloku 04 ho využijeme, aby časování fungovalo i přes přetečení `millis()`.

Kvíz 3 – proč unsigned long?

Takhle se bude celý rok psát měření času. Proč ne int?

```
unsigned long start = millis();
```

- A) Protože millis() vrací desetinné číslo.
- B) Protože čas nemůže být negativní, jinak je to jedno.
- C) Protože int (2 bajty) by přetekl už po 32 sekundách.
- D) Protože unsigned long je na AVR rychlejší.

Kvíz 3 – proč unsigned long?

Takhle se bude celý rok psát měření času. Proč ne int?

```
unsigned long start = millis();
```

- A) Protože millis() vrací desetinné číslo.
- B) Protože čas nemůže být negativní, jinak je to jedno.
- C) **Protože int (2 bajty) by přetekl už po 32 sekundách.**
- D) Protože unsigned long je na AVR rychlejší.

C)

Do int se vejde 32 767 ms, tedy **32 sekund**. Do unsigned long 4 294 967 295 ms, tedy **asi 49 dní**.

Kvíz 4 – #define nebo const? (rozšíření)

Dva způsoby, jak pojmenovat pin. Který je lepší a proč?

```
#define PIN_LED 13           // varianta A  
const uint8_t PIN_LED = 13; // varianta B
```

- A) #define, protože nezabírá žádnou paměť.
- B) const, protože má typ a překladač ho zkontroluje.
- C) Je to úplně jedno, oba se přeloží stejně.
- D) const na mikrokontroleru nefunguje.

Kvíz 4 – #define nebo const? (rozšíření)

Dva způsoby, jak pojmenovat pin. Který je lepší a proč?

```
#define PIN_LED 13           // varianta A  
const uint8_t PIN_LED = 13; // varianta B
```

- A) #define, protože nezabírá žádnou paměť.
- B) **const, protože má typ a překladač ho zkontroluje.**
- C) Je to úplně jedno, oba se přeloží stejně.
- D) const na mikrokontroleru nefunguje.

B)

#define je jen **textová náhrada** před překladem, překladač o typu nic neví. **Proč na tom záleží – hned na dalším slidu.**

Bez typu je to jen text (rozšíření)

```
#define SOUCET 2+3
uint8_t x = 10 * SOUCET;    // 23 ?!

// prekladac dosadil text a pocita:
// 10 * 2+3 -> (10*2)+3 -> 23

const uint8_t SOUCET = 2+3;
uint8_t y = 10 * SOUCET;    // 50
```

`#define` dosadí **znaky**, ne hodnotu. Priorita operátorů se pak spočítá jinak, než jste mysleli – a nic to nezahlásí.

`const` má hodnotu **i typ**. Překladač ji spočítá jednou a dál s ní zachází jako s číslem.

Druhá nepříjemnost: v chybovém hlášení po `#define` uvidíte dosazenou hodnotu, ne jméno konstanty. Chybu pak hledáte podle čísla 13 místo podle `PIN_LED`.

Špatný typ = tiché přetečení (rozšíření)

Klasická chyba, kterou letos uvidíme v bloku 07 – převod z ADC na milivoly:

```
uint16_t adc = analogRead(A0);    // 0 az 1023

// CHYBA: 1023 * 5000 = 5 115 000 se do 16 bitu nevejde
uint16_t mV      = adc * 5000 / 1023;

// OPRAVA: první cinitel do 32 bitu -> celý výpočet ve 32 bitech
uint32_t mV_ok   = (uint32_t)adc * 5000 / 1023;
```

Mezivýsledek `adc * 5000` se počítá v **16 bitech**, protože oba operandy se vejdou do `int`. Do 16 bitů se vejde 32 767 – pět milionů tedy přeteče a vyjde nesmysl.

Přetypování prvního činitele na `uint32_t` přinutí překladač počítat celý výraz ve 32 bitech.

Proč je to zákeřné

Program se přeloží, nahraje a běží. Jen čidlo ukazuje divná čísla – a hledá se to hůř než chyba, která rovnou spadne.

Souvislost s kvízem 1: na 32bitovém ARM by ten samý řádek fungoval.

Co dává kontrola typu překladačem (rozšíření)

Překladač umí	S const (má typ)	S #define (jen text)
varovat při zúžení hodnoty	ano	ne
zvolit šířku výpočtu	podle typu	podle dosazeného textu
ohlídat záměnu znaku a čísla	ano	ne
ukázat v chybě jméno	ano	ne, ukáže hodnotu

Shrnuto: datový typ není formalita, ale **informace pro překladač**, kolik bitů má na výpočet použít a co je ještě povolené. Kde typ chybí, překladač nemá co kontrolovat – a chyba se místo hlášení projeví až divným chováním desky.

K čemu je preprocesor naopak nenahraditelný (rozšíření)

```
#define DEBUG    // zakomentovat = vypnout

#ifdef DEBUG
    #define LOG(x)  Serial.println(x)
#else
    #define LOG(x)          // vubec nic
#endif

void loop() {
    LOG(adc);    // v ostre verzi zmizi
}
```

Co tím získám

Ladicí výpisy zapnu a vypnu **jedním řádkem**. Ve vypnuté verzi se ty řádky **vůbec nepřeloží** – nezaberou Flash, nezaberou SRAM a nezdržují.

`const` tohle nesvede: proměnná vždycky existuje a `if` se vždycky vyhodnocuje **za běhu**. Makro může expandovat na **nic**.

Na AVR to není maličkost: každý ladicí `Serial.println("...")` je řetězec, který zabírá Flash i SRAM. Deset takových a 2 kB jsou pryč.

Další práce pro preprocesor (rozšíření)

Přepnutí podle platformy

```
#if defined(__AVR__)
    const uint8_t PIN_LED = 13;
#elif defined(ARDUINO_ARCH_RP2040)
    const uint8_t PIN_LED = 25;
#endif
```

Stejná skica se přeloží pro UNO i pro Pico. Souvisí s tématem 01 – každá platforma má jiná čísla pinů.

Dělicí čára: preprocesor rozhoduje, **které řádky se vůbec přeloží** – to proměnná nikdy nesvede. Ale **pojmenovat hodnotu** má const, protože u toho chceme typ a kontrolu.

Ochrana hlavičkového souboru

```
#ifndef MOTOR_H
#define MOTOR_H
    // obsah hlavičky
#endif
```

Zabrání tomu, aby se hlavička vložila dvakrát. Bez toho překladač hlásí „redefinition“. Použijeme v bloku 15 u vlastní knihovny.

Řídicí struktury

Větvení – if / else if / else

```
if (teplota > 30) {  
    zapniVentilator();  
} else if (teplota > 25) {  
    zapniSlabe();  
} else {  
    vypni();  
}
```

Podmínky se zkouší **shora dolů** a provede se **jen první**, která platí. Složené závorky se dají u jednoho příkazu vynechat – **nedělejte to**, je to zdroj chyb.

Relační	Logické
== rovná se	&& a zároveň
!= nerovná se	nebo
< >	! negace
<= >=	

Drobnosti, které se hodí: $i \% 2 == 0$ – sudé číslo;
 $\text{max} = (a > b) ? a : b$; – ternární operátor, zkrácený zápis if-else.

Kvíz 5 – co se vypíše?

```
uint8_t x = 0;  
  
if (x = 5) {  
    Serial.println("ANO");  
}  
Serial.println(x);
```

- A) Nevypíše se nic, x je 0.
- B) Vypíše ANO a pak 5.
- C) Překladač zahlásí chybu.
- D) Vypíše ANO a pak 0.

Kvíz 5 – co se vypíše?

```
uint8_t x = 0;  
  
if (x = 5) {  
    Serial.println("ANO");  
}  
Serial.println(x);
```

- A) Nevypíše se nic, x je 0.
- B) **Vypíše ANO a pak 5.**
- C) Překladač zahlásí chybu.
- D) Vypíše ANO a pak 0.

B) – klasická past: = není ==

x = 5 je **přiřazení**; jeho výsledek 5 je nenulový, tedy „pravda“. Překladač to nejvýš **varuje**.

switch – výběr z výčtu hodnot (rozšíření)

```
switch (prikaz) {  
    case 'r':  
        rozsvit();  
        break;  
    case 'z':  
        zhasni();  
        break;  
    default:  
        Serial.println("?");  
}
```

Kdy switch a kdy else if

switch – jedna proměnná, porovnává se s **konkrétními** hodnotami (znak, číslo režimu).

else if – rozsahy (> 30) a složené podmínky.

Pozor: switch umí jen celočíselné typy a char, ne float ani text.

Kam to povede: v bloku 05 z toho bude **stavový automat** – switch nad proměnnou stav.

Kvíz 6 – co se vypíše? (rozšíření)

```
uint8_t rezim = 1;

switch (rezim) {
    case 1: Serial.println("JEDEN");
    case 2: Serial.println("DVA"); break;
    case 3: Serial.println("TRI"); break;
}
```

- A) JEDEN B) JEDEN a DVA C) všechny tři D) chyba překladu

Kvíz 6 – co se vypíše? (rozšíření)

```
uint8_t rezim = 1;

switch (rezim) {
    case 1: Serial.println("JEDEEN");
    case 2: Serial.println("DVA"); break;
    case 3: Serial.println("TRI"); break;
}
```

- A) JEDEEN **B) JEDEEN a DVA** C) všechny tři D) chyba překladu

B) – u case 1 chybí break

Bez break program **propadne** do dalšího case a pokračuje, dokud na nějaký break nenarazí.

Cykly – tři tvary, jeden účel

```
// znam pocet opakovani
for (uint8_t i = 0; i < 8; i++) {
    digitalWrite(i, HIGH);
}

// dokud plati podminka
while (mereni < 10) { ... }

// alespon jeden prubeh
do { ... } while (mereni < 10);
```

Cyklus	Kdy ho použít
for	počet opakování znám dopředu
while	nevím; nemusí se provést ani jednou
do-while	provede se vždy aspoň jednou

Na Arduino navíc: `loop()` je sám nekonečný cyklus. Vnější `while (1)` už tedy psát nemusíte – máte ho zdarma. `break` cyklus ukončí, `continue` přeskočí na další průběh.

Kvíz 7 – kolik řádků se vypíše?

```
for (uint8_t i = 1; i <= 8; i = i + 2) {  
    Serial.println(i);  
}
```

- A) 3 B) 4 C) 8 D) nekonečně mnoho

Kvíz 7 – kolik řádků se vypíše?

```
for (uint8_t i = 1; i <= 8; i = i + 2) {  
    Serial.println(i);  
}
```

- A) 3 **B) 4** C) 8 D) nekonečně mnoho

B) – vypíše se 1, 3, 5, 7

Krok není `i++`, ale `i = i + 2`. Při `i = 9` podmínka `i <= 8` přestane platit a cyklus skončí.

Tři věci, na které se u cyklu vždycky podívat: odkud se počítá (`i = 0` nebo `1`), jaká je podmínka (`<` nebo `<=`) a jaký je krok. Chyba v kterékoliv z nich je „o jedno vedle“.

Na mikrokontroleru je čekání v cyklu past

```
// PAST: dokud se drzi tlacitko,  
// deska nedela NIC jineho  
while (digitalRead(PIN_TL) == LOW) {  
    // cekame...  
}  
  
// PAST: totez, jen jinak napsane  
delay(2000);
```

Na PC je čekání v cyklu neškodné – operační systém mezitím obsluhuje všechno ostatní.

Na mikrokontroleru **není nikdo, kdo by to udělal za vás**. Když program čeká, deska nečte ostatní tlačítka, nepřepíná LED a nekomunikuje.

Kam to povede: od bloku 04 je `delay()` v zadáních **zakázaný**. Místo čekání se v `loop()` jen podíváme, kolik je `millis()`, a hned se vrátíme.

Program bez konce a výstup

Kvíz 8 – kde je v Arduinu `main()`?

Na PC program skončí, když `main()` dojde k `return 0;`. Co dělá Arduino?

- A) `main()` tam není, Arduino žádný nemá.
- B) `main()` tam je, jen ho nevidíme – volá `setup()` a pak `loop()` v nekonečném cyklu.
- C) `setup()` a `loop()` se volají střídavě po každé instrukci.
- D) `loop()` se zavolá jen tehdy, když se něco změní na pinu.

Kvíz 8 – kde je v Arduinu `main()`?

Na PC program skončí, když `main()` dojde k `return 0;`. Co dělá Arduino?

- A) `main()` tam není, Arduino žádný nemá.
- B) `main()` tam je, jen ho nevidíme – volá `setup()` a pak `loop()` v nekonečném cyklu.
- C) `setup()` a `loop()` se volají střídavě po každé instrukci.
- D) `loop()` se zavolá jen tehdy, když se něco změní na pinu.

```
int main(void) {    // tohle za nas doplni Arduino
  init(); setup();    // probehne jednou
  for (;;) { loop(); }    // pusti se porad znovu
}
```

Důsledek: `loop()` běží tisíce krát za sekundu

Na PC	Na Arduino
<code>printf()</code>	<code>Serial.print()</code>
<code>scanf()</code>	<code>Serial.read()</code> (jinak!)
<code>malloc()</code>	nepoužíváme
<code>exit()</code>	není kam skončit
soubory	nejsou (bez SD karty)
výjimka, chyba	reset nebo zamrznutí

Hlavní past roku: `delay(1000)` znamená, že deska celou sekundu **nedělá nic jiného**.

obrázek k dodání

Vedle sebe: konzole OnlineGDB
s výstupem `printf` a sériový
monitor Arduino IDE s výstupem
`Serial.println` – screenshot

`../obrazky/02_konzole_vs_serial`

`Serial.read()` **nečeká**, až něco napíšete. Vrátí to, co už přišlo, nebo `-1`.

Z printf na Serial.print (rozšíření)

Na PC	Na Arduinu
printf("%d\n", x)	Serial.println(x)
printf("%c", z)	Serial.print(z)
printf("%s", t)	Serial.print(t)
printf("%.2f", f)	Serial.print(f, 2)
printf("%x", x)	Serial.print(x, HEX)

```
// printf("a=%d b=%d\n", a, b);  
Serial.print("a=");  
Serial.print(a);  
Serial.print(" b=");  
Serial.println(b);
```

Tři věci, které se pokaždé zapomenou: `Serial.begin(9600)` v `setup()`; stejná rychlost nastavená i v sériovém monitoru; a `println` místo `print`, jinak se celý výstup slije na jeden řádek.

Lokální a globální proměnná (rozšíření)

```
uint8_t pocet = 0;    // globalni

void loop() {
    uint8_t docasna = 0; // lokalni
    pocet = pocet + 1;    // pamatuje si
    docasna = docasna + 1; // vzdy 1
}
```

Lokální proměnná při každém průběhu `loop()` **vznikne znovu od nuly**. Globální si hodnotu pamatuje.

Na PC se globální proměnné nedoporučují.
Na mikrokontroleru jsou běžné a v pořádku – stav zařízení musí přežít mezi průběhy `loop()`.

Kam to povede: v bloku 03 se naučíme `static` – lokální proměnnou, která si hodnotu pamatuje, ale nikdo jiný do ní nevidí.

Funkce, pole a paměť

Kvíz 9 – co se vypíše?

```
void zdvojnásob(int cislo) {  
    cislo = cislo * 2;  
}  
  
void setup() {  
    int a = 5;  
    zdvojnásob(a);  
    Serial.println(a);    // co se vypíše?  
}
```

- A) 10 B) 5 C) 0 D) nepřeloží se

Kvíz 9 – co se vypíše?

```
void zdvojnásob(int cislo) {  
    cislo = cislo * 2;  
}  
  
void setup() {  
    int a = 5;  
    zdvojnásob(a);  
    Serial.println(a);    // co se vypíše?  
}
```

- A) 10 **B) 5** C) 0 D) nepřeloží se

B) – parametr se předává kopií

Funkce zdvojnásobila kopii a kopie zanikla. Kdo chce změnit proměnnou volajícího, musí ji returnnout, nebo poslat **ukazatel** (blok 06).

Kvíz 10 – co se stane na Arduino?

```
int hodnoty[5];  
  
for (uint8_t i = 0; i <= 5; i++) {    // pozor na <=  
    hodnoty[i] = analogRead(A0);  
}
```

- A) Překladač zahlásí „index out of range“.
- B) Program spadne s chybovým hlášením v sériovém monitoru.
- C) Přepíše se cizí paměť – program se chová divně nebo se deska resetuje.
- D) Nic, poslední hodnota se zahodí.

Kvíz 10 – co se stane na Arduino?

```
int hodnoty[5];  
  
for (uint8_t i = 0; i <= 5; i++) {    // pozor na <=  
    hodnoty[i] = analogRead(A0);  
}
```

- A) Překladač zahlásí „index out of range“.
- B) Program spadne s chybovým hlášením v sériovém monitoru.
- C) **Přepíše se cizí paměť – program se chová divně nebo se deska resetuje.**
- D) Nic, poslední hodnota se zahodí.



int hodnoty[5] v SRAM – C meze pole nekontroluje

Rozpočet na 2 kB SRAM

ATmega328P má **2048 bajtů** SRAM –
a v nich musí být **všechny** proměnné, pole
i zásobník volání funkcí.

Co	Zabere
<code>uint8_t stav;</code>	1 B
<code>int hodnoty[100];</code>	200 B
<code>float mereni[100];</code>	400 B
<code>char text[200];</code>	200 B
<code>Serial (buffery)</code>	~180 B

Typický zážitek

Program se přeloží a nahraje, ale deska
se restartuje nebo píše nesmysly. Příčina:
SRAM je plná a zásobník přepsal
proměnné.

Jak to poznat

Překladač to napíše sám:

`Global variables use 1834 bytes (89%).`

Nad 75 % je čas ubrat.

Kvíz 11 – bitové operace (rozšíření)

Co vyjde? `uint8_t x = 1 << 3;`

- A) 3 B) 4 C) 8 D) 13

Kvíz 11 – bitové operace (rozšíření)

Co vyjde? `uint8_t x = 1 << 3;`

- A) 3 B) 4 **C) 8** D) 13

C) – posun o 3 bity vlevo

`0b00000001` → `0b00001000` = 8. Posun vlevo o 1 bit je násobení dvěma.

Operace	Co dělá	K čemu letos
<code>a & b</code>	AND	„je tenhle bit nastavený?“
<code>a b</code>	OR	„nastav bit“
<code>a ^ b</code>	XOR	„překlop bit“ (blikání LED)
<code>~a</code>	negace	inverze všech bitů
<code>1 << n</code>	posun vlevo	maska pro n -tý pin

Když překladač zahlásí chybu (rozšíření)

obrázek k dodání

Screenshot Arduino IDE s chybovým hlášením
překladače – vyznačit číslo řádku a text chyby
`../obrazky/03_chyba_prekladace`

Postup, který platí celý rok

- 1 Přečti **první** chybu, ne poslední.
- 2 Najdi **číslo řádku** – chyba je na něm nebo o řádek výš.
- 3 Nejčastější tři: chybějící `;`, chybějící `}`, nedeklarovaná proměnná.
- 4 Oprav jednu věc a přelož znovu.

Pozor: „error“ znamená, že se nic nenahrálo – na desce běží pořád **starý** program. To mate.

Šest věcí k zapamatování

- 1 `int` má na AVR **2 bajty**. Pište `uint8_t`, `int16_t`, `uint32_t` – je hned vidět, co se vejde.
- 2 Typy `unsigned` **přetékají** bez varování. Čas vždycky do `unsigned long`.
- 3 **= není ==** a u `case` nesmí chybět `break`. Obojí se přeloží a chová se divně.
- 4 Program **nikdy neskončí**. `setup()` jednou, `loop()` pořád dokola – a **čekání v cyklu** zastaví celou desku.
- 5 C **nekontroluje meze pole** a SRAM má jen 2 kB. Za koncem pole je cizí proměnná.
- 6 Parametr funkce se předává **kopii**.

Hned po přestávce: na jakém stroji to vlastně běží – hardwarové platformy, AVR a ARM.